CareNavigator PH

System Defense and Project Documentation

A role-aware healthcare navigation and digital care platform for the Philippines

Project title	CareNavigator PH
Prepared for	System / project defense
Proponents	[Insert names]
Adviser / Panel	[Insert name or panel]
Date	02 September 2026

Defense-ready working draft based on the current repository implementation and local verification.

Contents

4. Updated System Flowchart and Architecture Diagram

5. Database Design (ERD / Schema)

6. Project Progress Report (Current Accomplishments)

7. Defense Conclusion

Appendix A. Source-of-Truth Locations

Executive Summary

CareNavigator PH is a Flutter mobile-and-web healthcare-navigation and digital-care platform. It unifies verified hospital and clinician discovery, safety-aware care guidance, guest consultation intake, authenticated patient and doctor workflows, hospital operations, and platform governance in one role-aware system backed by Supabase.

The current prototype demonstrates an end-to-end care pathway and a deliberate security model: UI guards support usability, typed repositories isolate data access, while JWT validation, Row Level Security, private Storage, constraints, triggers, and server-side authorization enforce access. The project is functionally substantial, but it is not represented as production-ready; live credentialed journeys, provider/device validation, formal compliance work, and visual-baseline review remain.

1. Project Title and Description

Project Title

CareNavigator PH: A Role-Aware Healthcare Navigation and Digital Care Platform

Project Description

CareNavigator PH helps users find an appropriate healthcare facility and continue through consultation, clinical documentation, communication, and follow-up without losing context. Guests can discover verified care and submit a protected consultation request. Patients can book and manage care, communicate with clinicians, and access authorized records. Doctors can manage schedules, consultations, structured checkups, prescriptions, laboratories, and review workflows. Hospital administrators operate assigned facilities, while super administrators manage platform governance.

The application uses one adaptive Flutter codebase. GoRouter manages deep links and role-based routing, Riverpod manages state and Realtime streams, typed Dart repositories form the data-access boundary, and Supabase provides Auth, PostgreSQL, RLS, Realtime, private Storage, RPCs, Edge Functions, triggers, and scheduled jobs. External integrations include Groq, Gmail SMTP, Jitsi, OpenStreetMap, and OSRM.

2. Project Objectives Defined

General Objective

To design and implement a secure, responsive, role-aware healthcare platform that helps people discover appropriate care and supports the care lifecycle across authorized guests, patients, clinicians, hospital personnel, and platform administrators.

Specific Objectives

- Provide searchable, verified hospital, service, clinician, schedule, availability, map, and route information.

- Provide safety-aware preliminary guidance that detects emergency signals and directs users to appropriate care without presenting AI output as a diagnosis.
- Connect guest consultation intake, verification, registered-patient booking, consultation, messaging, document processing, and follow-up.
- Provide role-specific workspaces and actions for patients, doctors, hospital administrators, and super administrators.
- Protect clinical and identity data through backend-enforced authorization, private file storage, short-lived access, clinical invariants, and auditing.
- Support maintainability through a layered architecture, typed models and repositories, centralized design components, migrations, and automated tests.
- Provide evidence-based project reporting that distinguishes implemented work from remaining production validation.

3. Scope and Limitations Identified

Scope

Area	Included capabilities
Public access	Hospital and clinician discovery, maps/routes, assistant guidance, guest consultation request and verification
Patient care	Booking, rescheduling/cancellation, consultations, messages, notifications, records, prescriptions, diagnostics, files, profile and preferences
Doctor care	Patient relationships, scheduling, consultation lifecycle, structured notes/checkups, prescriptions, laboratory requests/results, document review
Hospital operations	Appointments, staff, departments, services, facilities, beds/capacity, emergency-room status, reports and audit views
Platform governance	Hospital approval, accounts, permissions, settings, analytics, security, maintenance and audit
Technical platform	Flutter, GoRouter, Riverpod, typed repositories, Supabase Auth/PostgreSQL/RLS/Realtime/Storage/RPC/Edge Functions/Cron
External integration	Groq AI, Gmail SMTP, Jitsi, OpenStreetMap and OSRM

Limitations

- Care-assistant and document-analysis outputs remain preliminary and must not be treated as a diagnosis, prescription, or replacement for professional care.
- External services require configured credentials, network access, and provider availability; those production side effects were not triggered for this defense document.

- The ordinary local test run skipped credential-dependent live Supabase journeys because live test variables and demo passwords were not supplied.
- The project still needs physical Android tests, an iOS/macOS build-and-device pass, load testing, backup/recovery drills, production monitoring, key rotation, and a formal privacy/security/compliance assessment.
- When public Supabase configuration is absent, the interface intentionally shows explicit unavailable states instead of fabricated hospital or clinical data.
- Eight visual regression cases currently differ from their approved golden images and require an intentional UI review and baseline decision.

4. Updated System Flowchart and Architecture Diagram

End-to-End System Flow

Figure 1. Core CareNavigator PH workflow from discovery through protected follow-up.

The flow preserves user and clinical context across discovery, guidance, consultation entry, scheduling, care delivery, documentation, and follow-up. At every transition, the system separates usability controls in Flutter from authoritative policy and data checks in Supabase.

Logical Architecture

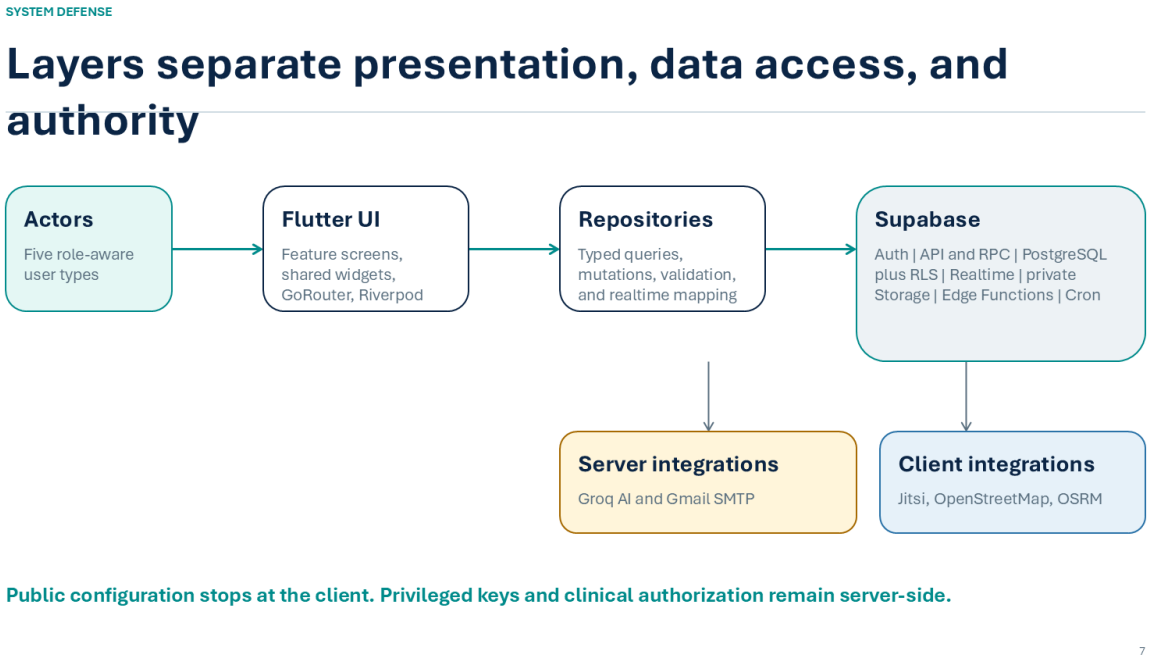

Figure 2. Layered client, repository, Supabase, and external-service architecture.

Editable Mermaid sources for VS Code are provided in `docs/defense/system_flowchart.mmd` and `docs/defense/system_architecture.mmd`. The companion `docs/defense/CareNavigator_PH_Diagrams.md` supports Markdown Preview with a Mermaid extension.

5. Database Design (ERD / Schema)

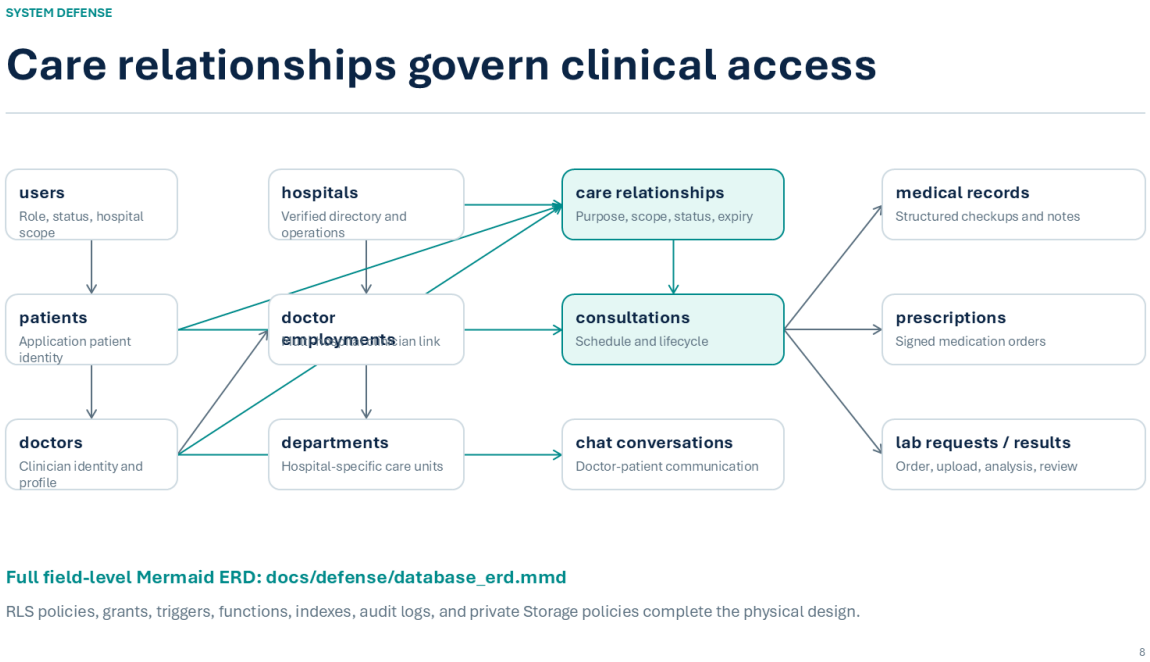

Figure 3. Defense-oriented conceptual ERD emphasizing identity, hospital, care-relationship, consultation, communication, and clinical-record domains.

The central design decision is to model care authorization as an explicit relationship among the patient, hospital, doctor, consultation, purpose, scope, status, and time boundary. This prevents a role label alone from granting broad clinical access. Multi-hospital identifiers and doctor employments preserve facility-specific context, while consultations produce structured records, prescriptions, laboratory orders/results, messages, documents, notifications, and audit evidence.

Database Design Principles

- Use UUID primary and foreign keys for authoritative entities and exact-ID actions.
- Use foreign keys, uniqueness constraints, check constraints, and clinical lifecycle triggers to preserve invariants.
- Enable Row Level Security for client-accessible tables and enforce patient, doctor, hospital, relationship, and action scope.
- Store sensitive clinical and identity documents in private buckets; issue short-lived signed URLs only after authorization.
- Use RPCs and Edge Functions for privileged, transactional, provider-facing, and auditable workflows.
- Use indexes for relationship lookups, status queues, schedules, recent activity, and conversation history.
- Preserve observability through status history, audit/security logs, processing jobs, and notification outboxes.

Mermaid ERD Source

Open `docs/defense/database_erd.mmd` in VS Code with a Mermaid extension. The file includes the main field-level entities and relationships used in the defense; the full physical schema remains defined by the Supabase database and the 63 migration files under `supabase/migrations/`.

6. Project Progress Report (Current Accomplishments)

Progress Snapshot

Workstream	Status	Current evidence	Remaining work
Architecture and UI foundation	**Implemented**	Adaptive Flutter shells; centralized theme/widgets; GoRouter; Riverpod; typed repositories	Continue accessibility/device validation
Public discovery and guest access	**Implemented**	Hospitals, clinicians, map/routing, assistant, request/verification flows, explicit unavailable states	Live provider and credentialed browser journeys
Patient and doctor care	**Implemented**	Booking, scheduling, consultation lifecycle, messaging, documents, checkups, prescriptions, diagnostics and labs	Live multi-role acceptance pass
Hospital and platform administration	**Implemented**	Hospital operations, staff/services/departments, capacity/ER, approvals, accounts, permissions, settings, maintenance and audits	Operational runbooks and production monitoring
Security and data model	**Implemented; review remains**	RLS-oriented access, private Storage, multi-hospital grants, constraints, triggers, RPCs and audit patterns	Independent security/privacy/compliance review
Quality verification	**Partially complete**	Static analysis clean; 243 tests passed; 11 credentialed tests skipped; 8 golden tests passed and 8 differed	Review visual changes; rerun live and device suites

Current Accomplishments

- Implemented one responsive Flutter application for web and mobile with public, patient, doctor, hospital-administrator, and super-administrator experiences.
- Established a repository-only Supabase boundary and role-aware routing/state architecture.
- Implemented public hospital and clinician discovery, OpenStreetMap/OSRM routing, guest intake, account flows, and explicit unavailable states.
- Implemented booking and consultation lifecycle workflows, secure Jitsi room access, Realtime messaging/notifications, and private file handling.
- Implemented structured checkups, prescriptions, diagnostic result extraction, laboratory requests/results, review states, and medication reminder workflows.
- Added multi-hospital identities, clinician employment, patient care relationships, grants/scopes, online consultation requests, and cross-source authorization.

- Added persistent patient conversations and persistent care-assistant conversation history.
- Added a `care-navigator-chat` Edge Function for safety-aware conversation, document extraction, hospital recommendation, and email-related workflows.
- Accumulated 63 focused Supabase migration files and seven SQL acceptance-test scripts in the repository.
- Verified the current code with `flutter analyze`: no issues found.

Latest Verification Results

Check	Result	Defense interpretation
`flutter analyze`	PASS - no issues	Static analysis is clean on the current repository state.
`flutter test`	243 passed; 11 skipped; 8 failed	Functional coverage is broad. Skips require live credentials. Failures are visual-golden differences.
Visual regression subset	8 passed; 8 failed	Hospital-admin and super-admin baselines passed; guest, patient, doctor, and prescription views require review.
Repository inventory	60 Dart implementation files; 39 Dart test files; 63 migration files; 7 SQL test files	The prototype has substantial breadth, but file counts are not substitutes for live acceptance.
Previous documented builds	Web release and Android debug passed in the August verification record	These are historical evidence and should be rerun before release.

Important: The project should not be described as having a completely passing test suite until the eight visual differences are reviewed and the credential-dependent tests are executed in an approved non-production environment.

Immediate Next Steps

- Review the eight visual diffs and approve or reject the corresponding UI changes before refreshing baselines.
- Run seeded, non-production patient, doctor, hospital-admin, and super-admin journeys with live Supabase configuration.
- Exercise Groq, SMTP, Jitsi, scheduled notification dispatch, private-file access, and mobile permissions in controlled environments.
- Run physical Android and iOS tests, browser console/interaction verification, load tests, and accessibility checks.
- Complete privacy, security, compliance, retention, backup, recovery, observability, incident-response, and key-rotation procedures.

7. Defense Conclusion

CareNavigator PH is a defensible integrated prototype: it addresses a clear healthcare-navigation problem, defines measurable objectives, covers the principal care and governance workflows, separates interface behavior from backend authority, and provides concrete implementation and test evidence. Its strongest architectural claim is not that one role can see everything, but that access is derived from verified identity, hospital context, care relationship, clinical purpose, and permitted action.

The recommended defense position is to approve continued development toward live end-to-end validation and production-readiness controls. The remaining gaps are documented as engineering and governance workâ€”not hidden as completed functionality.

Appendix A. Source-of-Truth Locations

Area	Repository location
Application entry and bootstrap	`lib/main.dart`; `lib/bootstrap.dart`
Routing and role guards	`lib/src/routing/app_router.dart`
State and providers	`lib/src/providers/`
Repository boundary	`lib/src/repositories/`
Domain models	`lib/src/models/`
Shared UI and theme	`lib/src/widgets/`; `lib/src/theme/`
Database migrations	`supabase/migrations/`
Edge Function	`supabase/functions/care-navigator-chat/`
Flutter and SQL tests	`test/`; `supabase/tests/`
Mermaid diagrams	`docs/defense/*.mmd`; `docs/defense/CareNavigator_PH_Diagrams.md`
Architecture and verification records	`SYSTEM_ARCHITECTURE.md`; `README.md`; `PRE_BUILD_REPORT.md`; `VERIFICATION_REPORT.md`

Verification date: 02 September 2026 (Asia/Manila). Counts and test results reflect the local repository state reviewed for this document. Team and adviser metadata remain placeholders because they were not present in the repository.